Hardware Software Interface (HSI)

INEM 2026-2027
Florent Bednarek

KNDS



Objectif Pédagogique

1. **Comprendre les abstractions qui existent entre le programme et le matériel sur lequel il est exécuté.**

2. **Devenir un développeur plus efficace :**
   a. **trouver et éliminer efficacement les bogue.**
   b. **Améliorer et comprendre ce qu'est la performance d'un software embarqué**

3. **S'initier à l'architecture software d'un système embarqué, en comprendre les enjeux.**

Programme Vendredi 23/10

Introduction

Cours 1 : Introduction aux Mémoires, Pointeurs et Allocation Dynamique

TP 1

Cours 2 : Représentation des Données Structurées & Alignement Mémoire

TP 2 partie 1

Cours 1 : Introduction aux Mémoires, Pointeurs et Allocation Dynamique

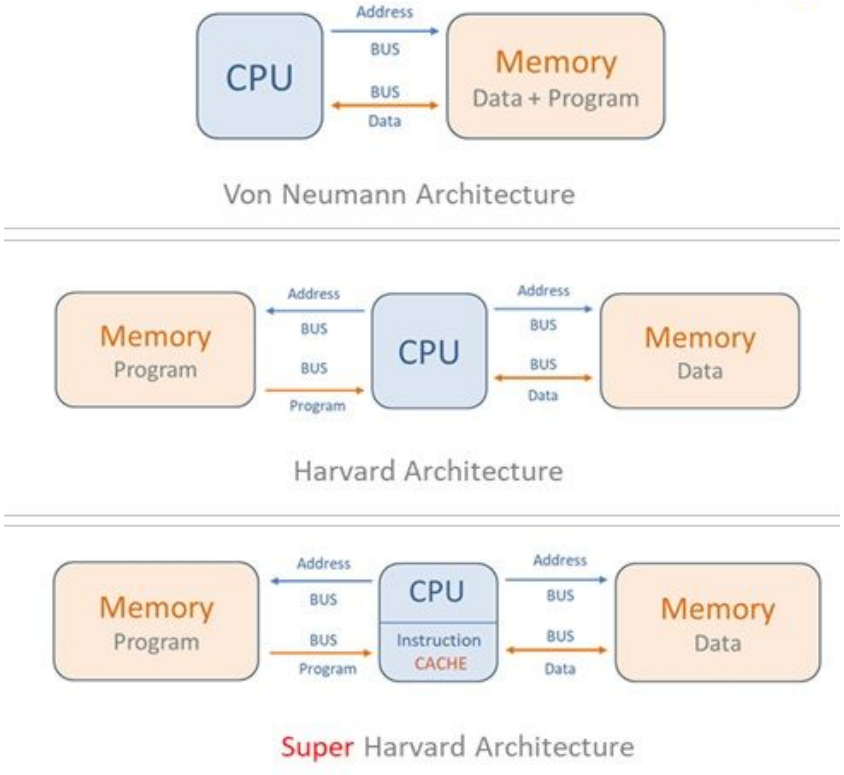
Représentation Binaire

Organisation et adressage de la mémoire

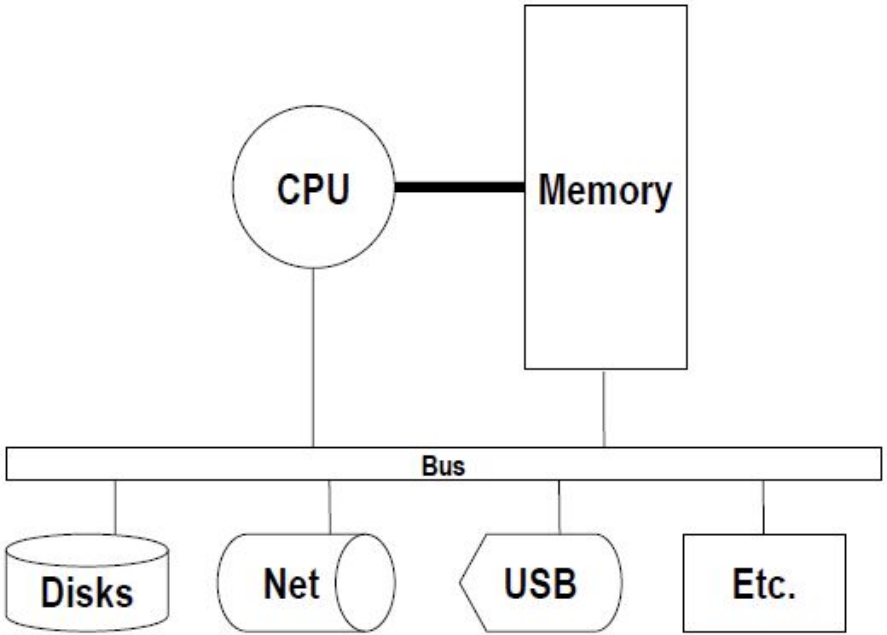
Manipulation de la mémoire et allocation dynamique

algèbre booléen et manipulation de bit

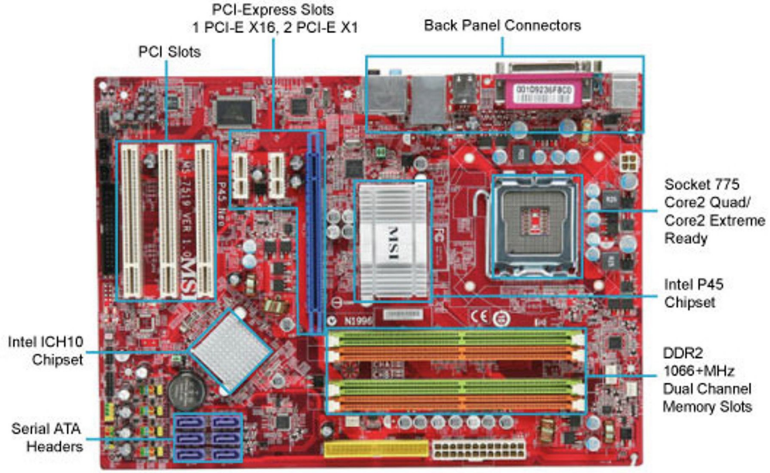
Hardware : Architecture processeur



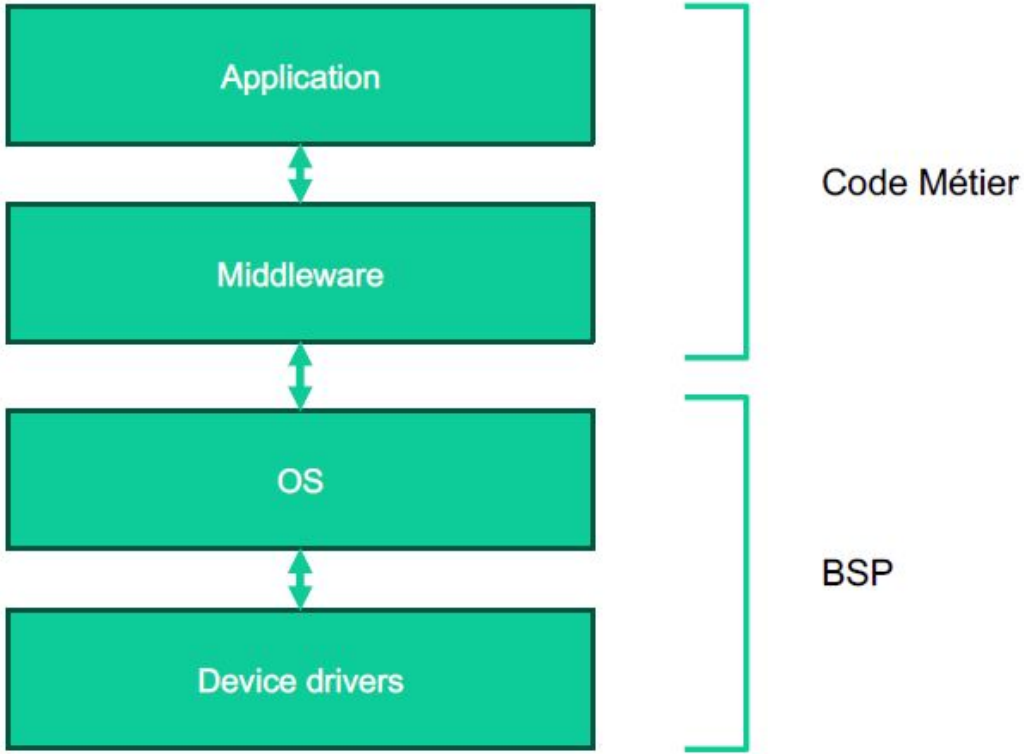
Hardware: vue logique



Hardware: vue physique



Représentation software



Ordre des Octets

Big endian (PowerPC, Sun, Internet)

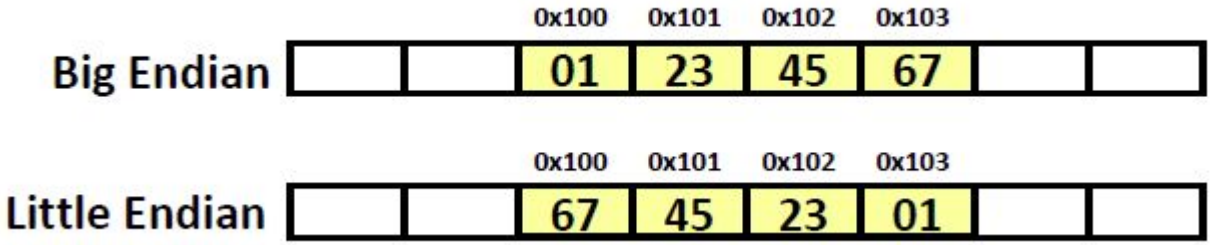
- **Bit de poid fort (MSB) à l'adresse la plus petite**

Little endian (x86)

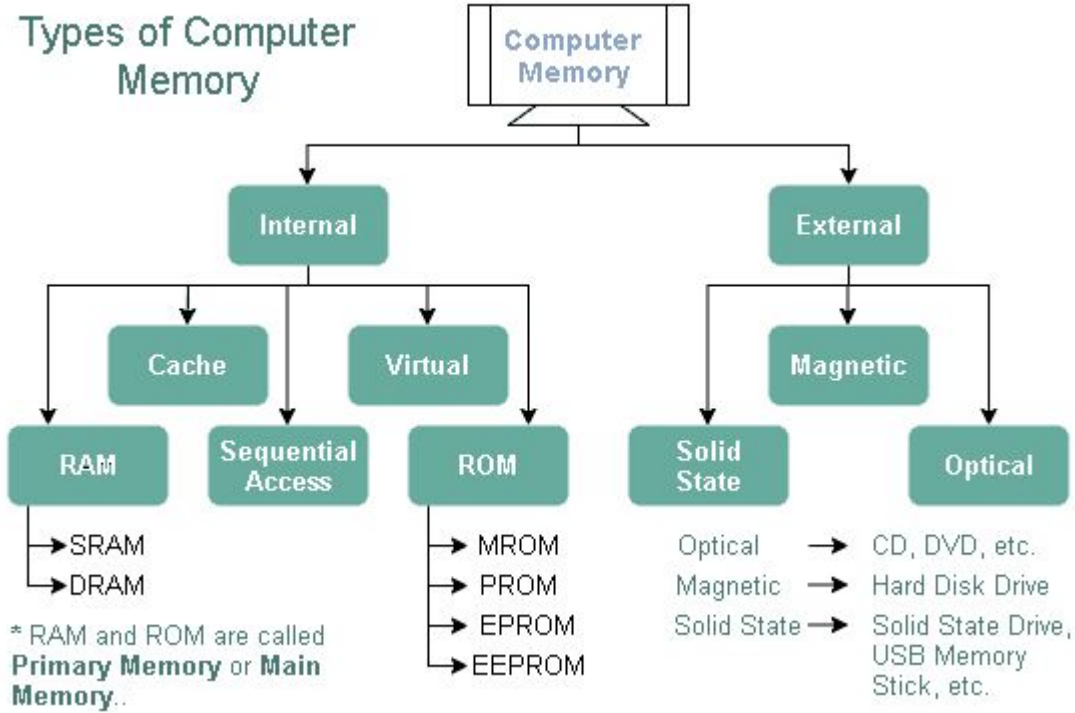
- **Bit de poid faible (LSB) à l'adresse la plus petite**

Exemple

- **Variable 0x01234567**
- **Adresse 0x100**

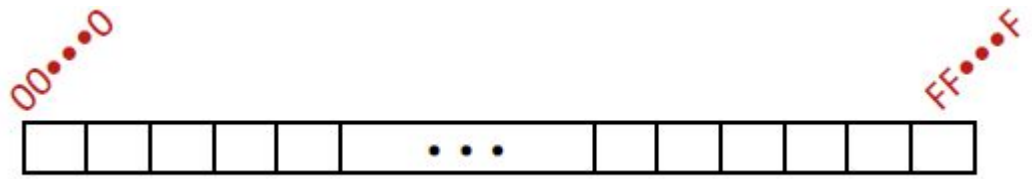


Organisation de la mémoire : Couche physique



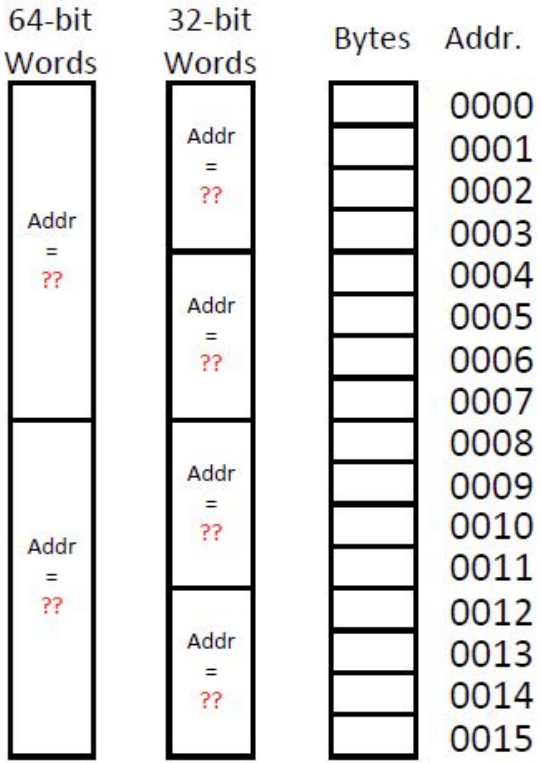
Exemple : Le CPU va à 3 GHz, la RAM va à 200 MHz (relativement). Le CPU passe son temps à attendre.

Organisation de la mémoire : Couche OS



- **MMU (Memory Management Unit) : Composant hardware qui traduit les adresses.**

- **Virtual Memory : Chaque processus croit avoir toute la mémoire (de 0x0000 à 0xFFFF...).**
  - ***Avantage :* Isolation (un crash ne tue pas l'OS).**
  - ***Coût :* TLB (Translation Lookaside Buffer) Miss. Si l'adresse n'est pas en cache TLB, le CPU doit parcourir la "Page Table" en RAM (lent).**



Organisation de la mémoire : Couche software

1. **.text (Code) :** Instructions en lecture seule. (Essayer d'écrire ici = Segfault).
2. **.data :** Variables globales/statiques initialisées.
3. **.bss :** Variables globales/statiques non initialisées (automatiquement à 0).
4. **Heap (Tas) :** Allocation dynamique (malloc).
5. **Stack (Pile) :** Variables locales, contextes de fonctions.

Stack et Heap

Caractéristique	Stack (Pile)	Heap (Tas)
Vitesse	Très rapide	Lent (algorithme d'allocation)
Durée de vie	Portée de la fonction	Jusqu'au free()
Risques	Stack Overflow (récursion infinie, gros buffers)	Fragmentation, Memory Leaks, Double Free
Utilisation	Variables locales, petits buffers	Grosses données, structures dynamiques

Pointeurs

Un pointeur n'est qu'une variable qui contient une adresse (généralement 32 ou 64 bits).

```
int *p = 0x1000;
p + 1; // Ne vaut pas 0x1001, mais 0x1004 (car int = 4 octets)
```

Void Pointer (void*)

Le pointeur universel. Il perd l'information de type et de taille. On ne peut pas faire d'arithmétique dessus sans le caster. Très utilisé pour les drivers génériques ou les allocateurs mémoire.

Char et string

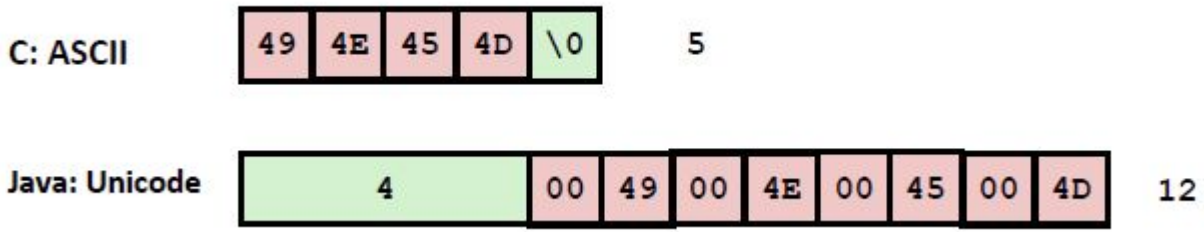
Caractère :

- **C : 1 octet ASCII**
- **Java : 2 octets Unicode**

String :

- **C : caractère null (‘\0’)**
- **Java : champs caché au début pour la longueur (4 octets)**

string ‘INEM’:



Char et string

Que fait la fonction strlen() en C ? Comment elle fonctionne

Utiliser toujours les fonctions strnlen, strncpy etc.
permet d’éviter les erreurs de segmentation

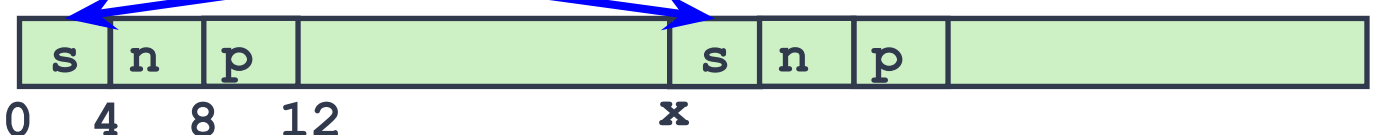
Casting et déplacements mémoire

On peut caster un pointer en n'importe quel autre pointer

```
struct BlockInfo {
     int sizeAndTags;
     struct BlockInfo* next;
     struct BlockInfo* prev;
};
typedef struct BlockInfo BlockInfo;
…
int x;
BlockInfo *b;
BlockInfo *newBlock;
…
newBlock = (BlockInfo *) ( (char *) b + x );
…
```

Cast b en char pour ajouter un offset sans scalling

Re-cast en BlockInfo pour pouvoir exploiter le pointeur de nouveau



Opérateurs logiques en C : &&, ||, !

Petit quiz :

- !0x41 → 0x00
- !0x00 → 0x01
- 0x69 && 0x55 → 0x01
- 0x00 && 0x55 → 0x00
- 0x69 || 0x55 → 0x01
- p && *p++ → if (p) { *p++;}

Opérateurs de bit en C : &, |, ^, ~

Des vecteurs de bits peuvent être utilisés pour représenter des ensembles :

```
01101001      {0, 3, 5, 6}
76543210

01010101      {0, 2, 4, 6}
76543210
```

Opérations

- & Intersection
- | Union
- ^ Différence symétrique
- ~ Complément

Allocation Dynamique

`*p = malloc(size)` assigne un bloc de mémoire et retourne un pointeur vers le premier octet

`*p = calloc(num, size)` assigne la mémoire pour un tableau et le rempli de 0

`free(p)` Libère le bloc mémoire associé à un pointeur

Allocation Dynamique : Schéma d’utilisation

```
int *ptr = malloc(10 * sizeof(int));

if (ptr == NULL) {

    return 1;
}

// Utilisation de la mémoire…

free(ptr);
ptr = NULL;
```

Allocation Dynamique : Risques

Fragmentation Mémoire externe / interne

Memory leak

Header d’allocation trop important

Temps d’allocation non déterministe

Use after `free` / dangling pointer

Attention aux `free` si `malloc` échoué

Allocation Dynamique : Solutions

Ne pas allouer dynamiquement !

Allocation Dynamique : Solutions

1. Allocation statique uniquement.
2. Allocation par pool de donnée à taille fixe.
3. Autorisation à l’init uniquement
4. Design pattern comme les Ring Buffer ou les Stack

Cours 2 : Données Structurées & Alignement Mémoire

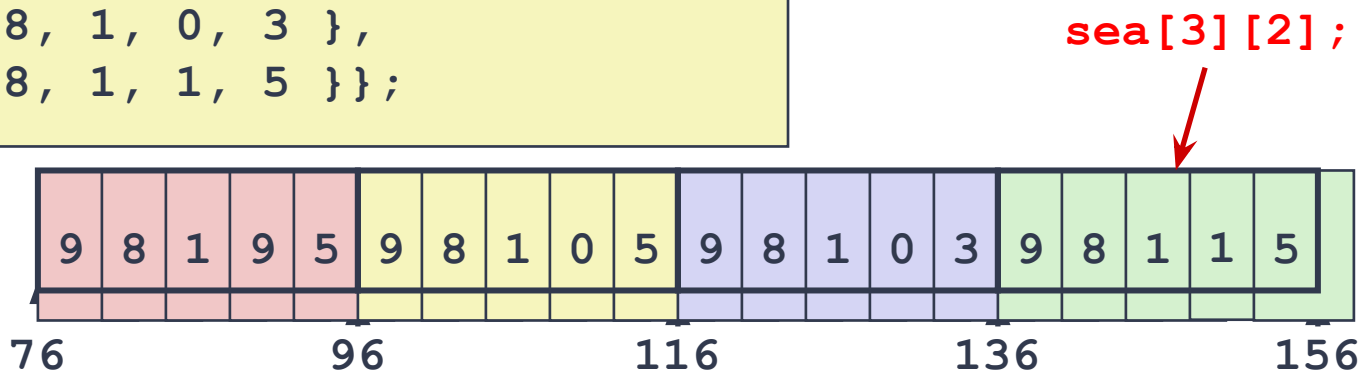
Tableau multi-dimensions vs Tableau multi-niveaux

Structures de données, Alignement

Analyse cartographie mémoire

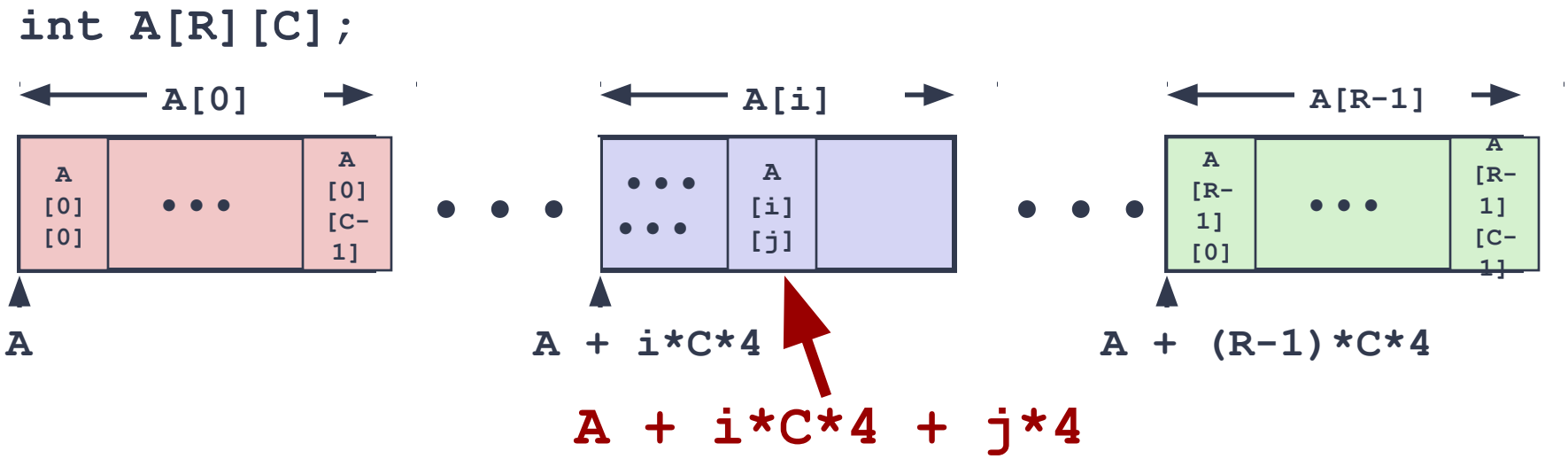
Tableaux multi-dimensions

```
int sea[4][5] =
  {{ 9, 8, 1, 9, 5 },
   { 9, 8, 1, 0, 5 },
   { 9, 8, 1, 0, 3 },
   { 9, 8, 1, 1, 5 }};
```



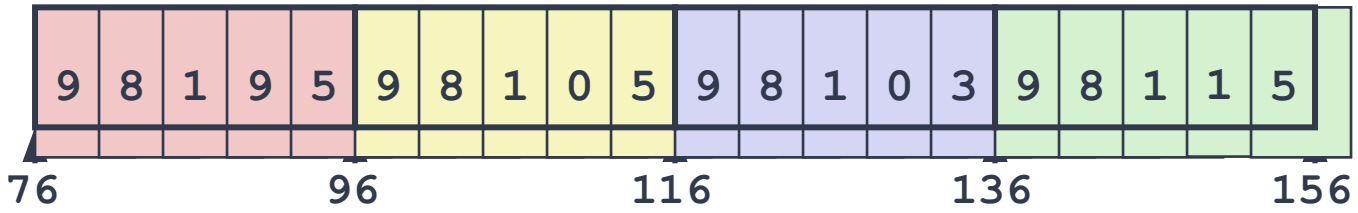
Les éléments sont organisés en “row-major” (garanti)

Tableaux multi-dimensions



Exemples de référence

```
int
sea[4][5];
```

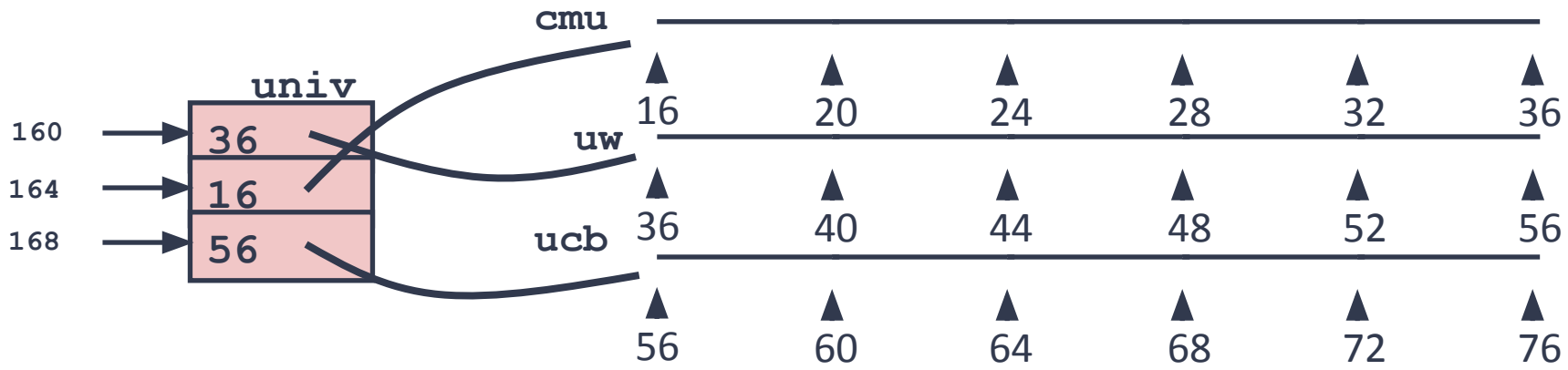


Référence	Adresse	Valeur	Garantie ?
sea[3][3]	76+20*3+4*3 = 148	1	Oui
sea[2][5]	76+20*2+4*5 = 136	9	Oui
sea[2][-1]	76+20*2+4*-1 = 112	5	Oui
sea[4][-1]	76+20*4+4*-1 = 152	5	Oui
sea[0][19]	76+20*0+4*19 = 152	5	Oui
sea[0][-1]	76+20*0+4*-1 = 72	??	Non

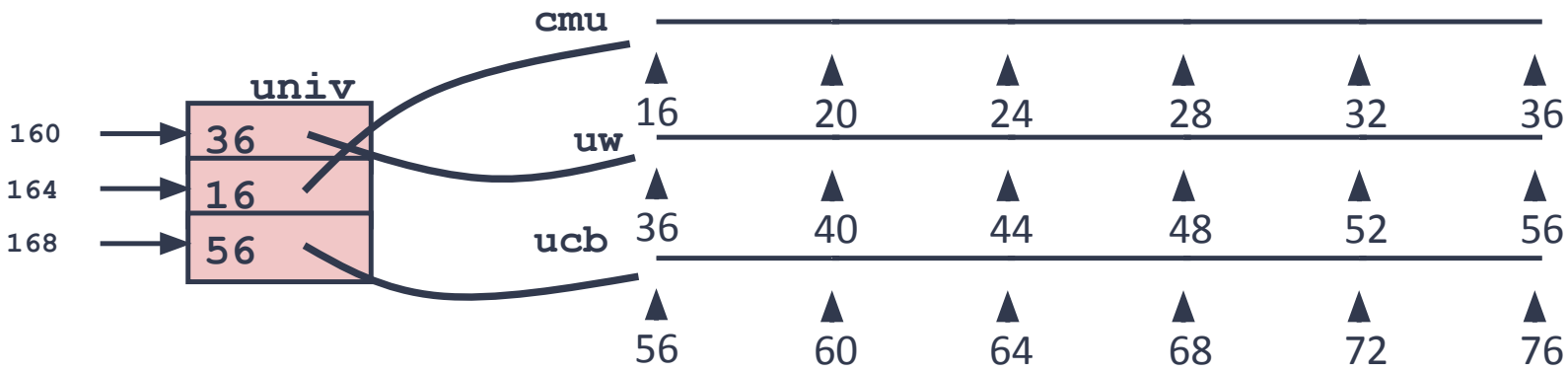
Tableaux multi-niveaux

```
int cmu[5] = { 1, 5, 2, 1, 3 };
int uw[5]  = { 9, 8, 1, 9, 5 };
int ucb[5] = { 9, 4, 7, 2, 0 };
```

```
int *univ[3] = {uw, cmu, ucb};
```



Exemples de référence



Référence	Adresse	Valeur	Garantie ?
`univ[2][3]`	`56+4*3 = 68`	`2`	**Oui**
`univ[1][5]`	`16+4*5 = 36`	`9`	**Non**
`univ[2][-1]`	`56+4*-1 = 52`	`5`	**Non**
`univ[3][-1]`	`??`	`??`	**Non**
`univ[1][12]`	`16+4*12 = 64`	`7`	**Non**

Tableau multi-dimensions vs Tableau multi-niveaux

```
int sea[4][5] =
  {{ 9, 8, 1, 9, 5 },
   { 9, 8, 1, 0, 5 },
   { 9, 8, 1, 0, 3 },
   { 9, 8, 1, 1, 5 }};
```

```
int cmu[5] = { 1, 5, 2, 1, 3 };
int uw[5]  = { 9, 8, 1, 9, 5 };
int ucb[5] = { 9, 4, 7, 2, 0 };
```

```
int *univ[3] = {uw, cmu, ucb};
```

Accès aux éléments

Tableau multi-dimensions

```
int get_sea_digit
   (int index, int dig)
{
  return sea[index][dig];
}
```

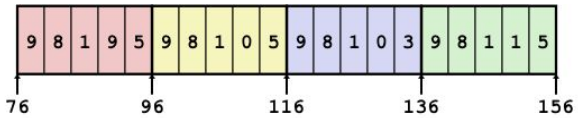
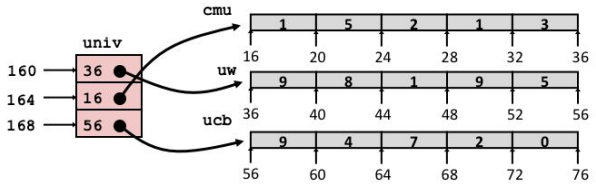


Tableau multi-niveaux

```
int get_univ_digit
   (int index, int dig)
{
  return univ[index][dig];
}
```



Accès différents !

`Mem[sea+20*index+4*dig]`

`Mem[Mem[univ+4*index]+4*dig]`

D’autres bizarreries du langage C :

https://zestedesavoir.com/articles/2505/les-bizarreries-du-langage-c-1/#1-les-operateurs-inusites

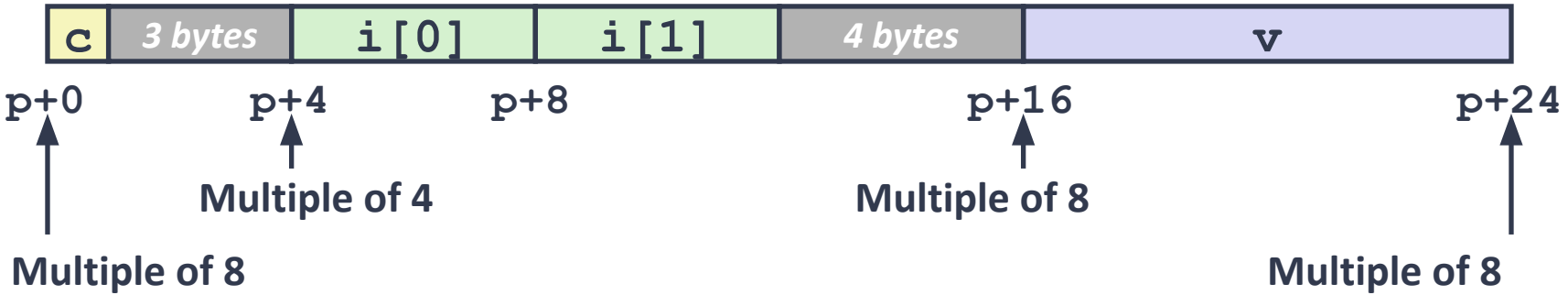
Structures & Alignement

Données non alignées




```
struct S1 {
  char c;
  int i[2];
  double v;
} *p;
```

Données alignées



Principes d’alignement

Données alignées

- Si le type primitif demande K octets, l’adresse doit être un multiple de K

Alignement de données est obligatoire sur certaines machines et « recommandé » sur IA32

- Traité différemment avec IA32 Linux, x86-64 Linux, et Windows!

Motivation

- Mémoire physique est accédée par « chunks » de 4 ou 8 octets, load et store plus efficace

Compilateur

- Insérer « padding » en structure pour assurer l’alignement correct des champs
- => Utiliser **sizeof()** pour avoir la vraie taille de structure !
- Utiliser –*Wpadded* dans (gcc) permet de lever un warning à chaque alignement de structure

Alignement avec structures

Dans la structure:

- Respecter l'exigence d'alignement de chaque membre

Placement de la structure

- Chaque structure a une exigence d'alignement K
  - K = alignement le plus grand
- Adresse initiale et longueur de la structure doivent être un multiple de K

```
struct S1 {
  char c;
  int i[2];
  double v;
} *p1;
```

Différentes conventions d'alignement

IA32 Windows ou x86-64:

- K = 8




```
struct S1 {
  char c;
  int i[2];
  double v;
} *p1;
```

IA32 Linux:

- K = 4



Sauver de la place !

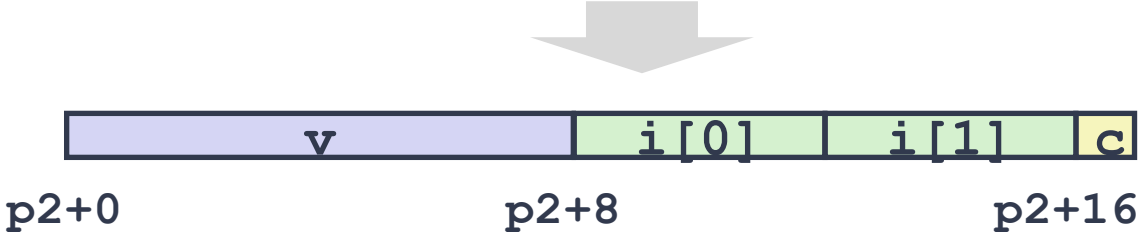
Mettre les données de type plus grand d'abord :

```
struct S1 {
  char c;
  int i[2];
  double v;
} *p1;
```

```
struct S2 {
  double v;
  int i[2];
  char c;
} *p2;
```

Effet (exemple x86-64, K=8)

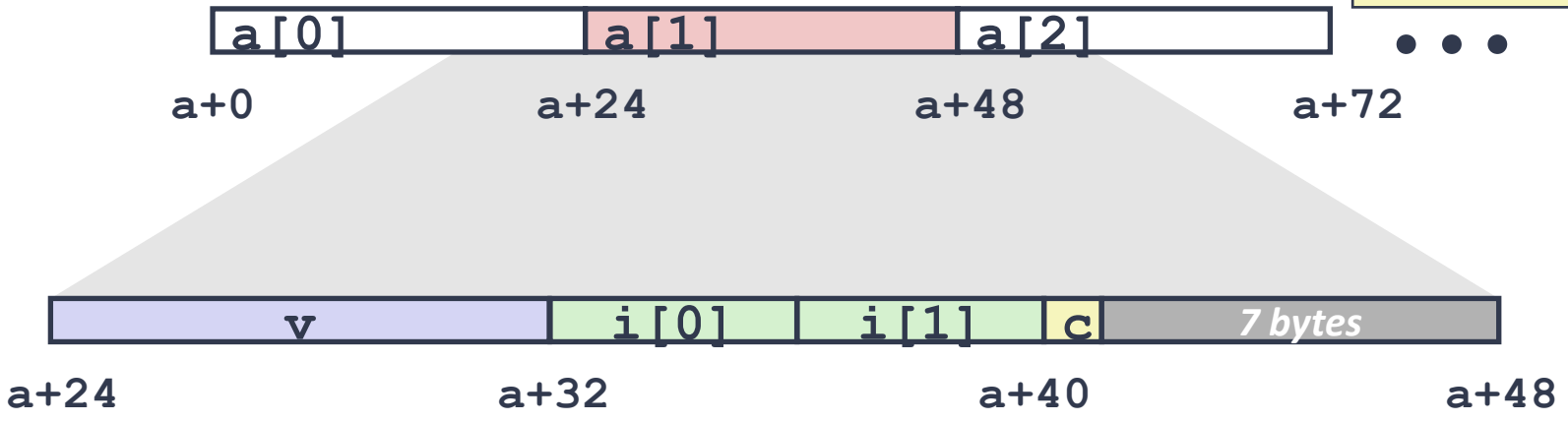




Malheureusement, la taille totale de la structure n'est pas un multiple de K

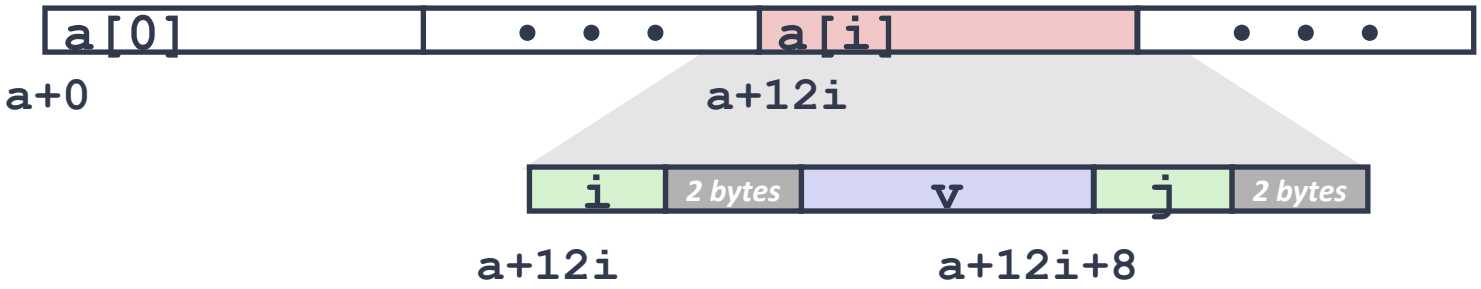
Tableau de structures

```
struct S2 {
  double v;
  int i[2];
  char c;
} a[10];
```



Un autre exemple

```
struct S3 {
   short i;
   float v;
   short j;
} a[10];
```



Taille et portabilité

Le type int est dangereux car sa taille dépend de l'architecture (16 bits sur Arduino, 32 bits sur ARM Cortex-M, 32 ou 64 bits sur x86_64).

- **La Règle d'Or :** Toujours utiliser <stdint.h> en programmation système.
  - uint8_t, int16_t, uint32_t : Vous contrôlez la taille exacte.
  - size_t : Pour tout ce qui touche aux index de boucles et tailles de mémoire (s'adapte à l'architecture).

Analyse cartographie mémoire

Le fichier .map est un compte rendu d'exécution du linker au format text ou HTML

Il details l’organisation de la mémoire pour la cible du programme.

Il comprend les section .text .bss .raw data et la correspondance des différents .o

Exemple : generation gcc -> `-Wl,-Map=output.map`

Analyse cartographie mémoire

```
#include <stdio.h>

int global_var = 42;

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

```
Archive member included because of file (symbol)

Linker script and memory map

Address of section .text set to 0x0000000000401000
Address of section .data set to 0x0000000000404000

.text           0x0000000000401000       0x17b
 *(.text.unlikely .text.unlikely.*)
 *(.text.startup .text.startup.*)
 .text.startup  0x0000000000401040       0x1c
/usr/lib/gcc/x86_64-linux-gnu/crti.o
                0x0000000000401040                main
 .text          0x0000000000401060       0x100 /tmp/ccABC123.o
                0x0000000000401060                main

.data           0x0000000000404000        0x14
 .data          0x0000000000404000         0x4 /tmp/ccABC123.o
                0x0000000000404000                global_var

.bss            0x0000000000404014         0x8
 *(.bss)
                0x000000000040401c                __bss_start
```

Analyse cartographie mémoire : pourquoi faire ?

Possibilité de chercher l’emplacement exact d’une donnée en mémoire.

Utile pour optimiser la taille d’un binaire

Check conformité matérielle (bootloader, table d'interruptions, MMIO)

Astuce : un résumé de la taille occupé en ram / flash est dispo tout en bas du fichier (l’utilitaire arm-none-eabi-size affiche le résumé sans .map)